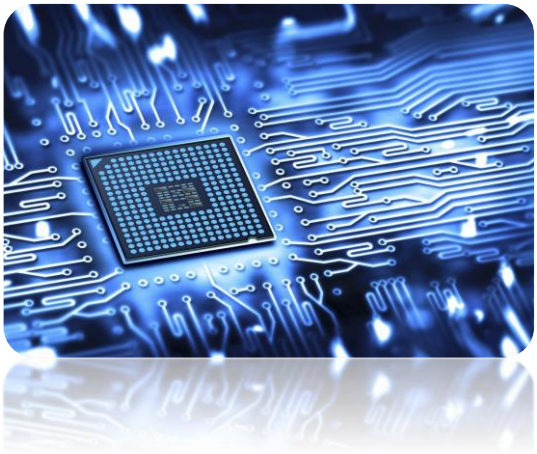
UNIVERSIDAD TECNOLOGICA CENTROAMERICANA
"Ceutec"



ARQUITECTURA DE COMPUTADORAS

Sección 276

- *Grupo de trabajo:* #1
- *Integrantes & números de cuentas:*

1. Alison Nicole Orellana León ---------- 62051020
2. Josué Abraham Ulloa ---------- 62211303
3. Kevin Hernández ---------- 61811772
4. Leonardo Aarón Martínez Molina ------- 22251055
5. Manuel Efraín Vásquez ---------- 62241016
6. Víctor David Lovo Zaldívar ---------- 62111478

- *Carrera:* Ingeniería Electrónica.

- *Ing.* Ricardo Adonis Caraccioli Abrego

- *Asunto*: Proyecto #1

Implementación del juego "2048" en pantalla OLED utilizando Arduino UNO.

- *Fecha*: 02/24/2026

- Hora de clase: 6:00 pm

  Martes/Jueves

Índice

Objetivo general:.....2
Objetivos especificos:.....2
Introduccion:.....3
Proyecto #1.....5
Implementación del juego “2048” en pantalla OLED utilizando Arduino UNO 5
Descripcion general del proyecto:.....5
Lista de materiales:.....5
Conexiones realizadas:.....6
Conexión de la pantalla OLED (I2C):.....6
Conexión del teclado matricial 4x4: 6
Problemas encontrados: 7
Codigo de programación:.....8
Anexos: 14
Montaje & simulación del juego “2048”:.....14
Grupo de trabajo #1: 17
Evidencia experimental (Video): 18
Enlace:.....18
Conclusiones: 19

Objetivo general:

- Diseñar e implementar el juego "2048" en una pantalla OLED utilizando una placa Arduino Uno, integrando un teclado matricial como entrada y aplicando principios de Arquitectura de Computadora.

Objetivos especificos:

- Crear una estructura de datos en forma de matriz 4x4 para representar el tablero del juego.
- Programar la lógica para el desplazamiento y la fusión de números en el juego 2048.
- Establecer la comunicación I2C entre el Arduino Uno y la pantalla OLED.
- Configurar y utilizar un teclado matricial 4x4 como dispositivo de entrada.
- Integrar todos los componentes electrónicos en una protoboard.
- Evaluar cómo se comporta el sistema frente a diferentes entradas del usuario.

Introduccion:

La asignatura de Arquitectura de Computadora no se limita únicamente al estudio teórico de la estructura interna de los sistemas digitales, sino que profundiza en la forma en que cada componente procesador, memoria, buses y dispositivos de entrada/salida se interrelaciona para ejecutar instrucciones y resolver problemas específicos. Comprender esta interacción entre hardware y software es esencial para cualquier estudiante de Ingeniería Electrónica, ya que permite visualizar cómo los conceptos abstractos vistos en clase se materializan en sistemas reales. Por esta razón, el desarrollo de proyectos prácticos representa una herramienta clave para afianzar el aprendizaje, ya que obliga a integrar conocimientos de programación, electrónica y diseño lógico en un solo sistema funcional.

En este contexto, el proyecto desarrollado consistió en la implementación del juego “2048” en una pantalla OLED de 0.96 pulgadas, empleando una placa Arduino Uno como unidad de procesamiento central. El sistema se complementó con un teclado matricial 4x4 para la interacción con el usuario, y todos los componentes fueron ensamblados sobre una protoboard mediante cables de conexión, lo que permitió una construcción organizada y flexible del circuito. La elección de este proyecto no fue aleatoria; se buscó diseñar una aplicación que implicara lógica estructurada, manipulación de datos en memoria y control de periféricos, manteniendo al mismo tiempo un nivel de complejidad adecuado para el curso.

El juego 2048, conocido por su dinámica basada en el desplazamiento y combinación de valores numéricos dentro de una matriz de 4x4, constituye un excelente ejemplo de aplicación lógica que puede adaptarse a sistemas embebidos. Aunque a simple vista parece un juego sencillo, su implementación requiere una planificación cuidadosa de la estructura de datos y del algoritmo que gestiona los movimientos y fusiones. Trasladar este funcionamiento a un microcontrolador implica administrar eficientemente los recursos limitados de memoria y procesamiento disponibles en el Arduino Uno.

Desde el punto de vista de arquitectura computacional, el proyecto involucró diversos aspectos fundamentales. En primer lugar, fue necesario diseñar una estructura de memoria que representara el tablero del juego mediante una matriz bidimensional, permitiendo almacenar y actualizar los valores dinámicamente. En segundo lugar, se implementó un sistema de procesamiento lógico capaz de detectar movimientos válidos, realizar fusiones entre valores iguales y actualizar el puntaje correspondiente. Asimismo, se incorporó un mecanismo de generación pseudoaleatoria para la aparición de nuevos valores en posiciones vacías del tablero, elemento esencial para la dinámica del juego.

Adicionalmente, el proyecto exigió el manejo de una interfaz gráfica en tiempo real a través de la comunicación I2C con la pantalla OLED, lo que implicó comprender el funcionamiento del protocolo y la forma en que el microcontrolador envía datos para ser representados visualmente. Paralelamente, fue necesario gestionar correctamente las

entradas digitales provenientes del teclado matricial, interpretando las señales de filas y columnas para identificar la tecla presionada y ejecutar la acción correspondiente.

En conjunto, este trabajo permitió aplicar de manera práctica conceptos como estructuras de datos, control de flujo, comunicación serial I2C, manejo de periféricos y diseño de sistemas interactivos. Más allá del resultado final, el proyecto representó una experiencia integral donde se evidenció cómo la arquitectura de una computadora —aunque en este caso representada por un microcontrolador— cobra sentido cuando el hardware y el software operan de forma coordinada para cumplir una función específica.

Proyecto #1

Implementación del juego "2048" en pantalla OLED utilizando Arduino UNO

Descripcion general del proyecto:

El proyecto consiste en la creación de una versión funcional del juego 2048 en un sistema embebido. El Arduino Uno actúa como unidad central de procesamiento, ejecutando el código que:

- Controla el tablero.
- Detecta las teclas presionadas.
- Realiza las operaciones lógicas.
- Actualiza la pantalla OLED.
- Calcula el puntaje.
- Detecta el fin del juego.

El tablero del juego se representa mediante una matriz de 4x4 posiciones almacenadas en memoria. Cada celda guarda el exponente del número (por ejemplo, 1 representa 2, 2 representa 4, 3 representa 8, etc.), lo que optimiza el uso de memoria.

El usuario controla el juego mediante el teclado 4x4 utilizando las teclas:

- 2 → Arriba
- 8 → Abajo
- 4 → Izquierda
- 6 → Derecha
- 9 → Reiniciar juego

La pantalla OLED muestra:

- El nombre del juego.
- El puntaje acumulado.
- La cuadrícula del tablero.
- Los valores actuales.
- Mensajes como "GAME OVER" o "NO MOVE".

Lista de materiales:

Arduino Uno	1
Pantalla OLED 0.96"	1
Teclado 4x4	1

Protoboard	1
Cables jumper macho-macho	
Cable USB para programación	
Computadora con Arduino IDE	
Librerías: • Adafruit_GFX • Adafruit_SSD1306 • Keypad	

Conexiones realizadas:

Conexión de la pantalla OLED (I2C):

La pantalla OLED funciona mediante protocolo I2C, utilizando únicamente 4 pines:

OLED → Arduino Uno

- VCC → 5V
- GND → GND
- SDA → A4
- SCL → A5

Es importante destacar que inicialmente se presentó un problema porque la pantalla estaba siendo alimentada incorrectamente. Al corregir la alimentación y verificar la dirección I2C (0x3C), el módulo funcionó correctamente.

Conexión del teclado matricial 4x4:

El teclado tiene 8 pines: 4 filas y 4 columnas.

Conexiones según el código:

Filas:

- R1 → Pin 10
- R2 → Pin 8
- R3 → Pin 7
- R4 → Pin 6

Columnas:

- C1 → Pin 5
- C2 → Pin 4
- C3 → Pin 3
- C4 → Pin 2

El Arduino escanea constantemente las filas y columnas para detectar qué tecla ha sido presionada.

Problemas encontrados:

Durante el desarrollo del proyecto surgieron varios inconvenientes:

1. **Problemas con los comandos asignados al teclado:** Inicialmente los movimientos no coincidían con la dirección esperada. Esto se resolvió ajustando correctamente la correspondencia entre teclas y funciones de movimiento.

2. **Problema con la pantalla OLED:** La pantalla no encendía debido a una alimentación incorrecta. Tras revisar el datasheet y verificar el voltaje adecuado, el problema fue solucionado.

3. **Decisión entre OLED y LCD:** Hubo indecisión sobre utilizar pantalla LCD o OLED. Finalmente se eligió OLED debido a:

- Mejor contraste.
- Menor consumo.
- Mejor calidad visual.
- Interfaz I2C más sencilla.

Todos los inconvenientes fueron resueltos satisfactoriamente, lo que fortaleció el aprendizaje práctico del equipo.

Codigo de programación:

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <Keypad.h>

// ================= OLED 128x64 SSD1306 I2C =================
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define OLED_RESET   -1
#define OLED_ADDR    0x3C   // si no enciende, prueba 0x3D

Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);

// ================= Keypad 4x4 =================
const byte ROWS = 4;
const byte COLS = 4;

char keys[ROWS][COLS] = {
  {'1','2','3','A'},
  {'4','5','6','B'},
  {'7','8','9','C'},
  {'*','0','#','D'}
};

// Pines sugeridos (NO uso el 9)
byte rowPins[ROWS] = {10, 8, 7, 6};   // R1,R2,R3,R4
byte colPins[COLS] = {5, 4, 3, 2};    // C1,C2,C3,C4

Keypad keypad = Keypad(makeKeymap(keys), rowPins, colPins, ROWS, COLS);

// ================= 2048 (tablero y puntaje) =================
// guardamos exponentes: 0=vacío, 1=2, 2=4, 3=8...
uint16_t board[4][4];
uint32_t score = 0;

// Random simple (sin librerías extra)
uint32_t rngState = 0xA5A5A5A5;
uint32_t xorshift32() {
  uint32_t x = rngState;
  x ^= x << 13;
  x ^= x >> 17;
  x ^= x << 5;
  rngState = x;
  return x;
}

uint16_t expToValue(uint16_t e) {
```

```
  if (e == 0) return 0;
  return (uint16_t)(1u << e); // 2^e
}

int emptyCount() {
  int c = 0;
  for (int r=0; r<4; r++)
    for (int c2=0; c2<4; c2++)
      if (board[r][c2] == 0) c++;
  return c;
}

void addRandomTile() {
  int empties = emptyCount();
  if (empties == 0) return;

  int pick = (int)(xorshift32() % empties);

  for (int r=0; r<4; r++) {
    for (int c=0; c<4; c++) {
      if (board[r][c] == 0) {
        if (pick == 0) {
          // 90% 2 (exp=1), 10% 4 (exp=2)
          board[r][c] = ((xorshift32() % 10) == 0) ? 2 : 1;
          return;
        }
        pick--;
      }
    }
  }
}

bool canMove() {
  if (emptyCount() > 0) return true;
  for (int r=0; r<4; r++) {
    for (int c=0; c<4; c++) {
      if (r < 3 && board[r][c] == board[r+1][c]) return true;
      if (c < 3 && board[r][c] == board[r][c+1]) return true;
    }
  }
  return false;
}

// ================= OLED Dibujo =================
void drawBoard() {
  display.clearDisplay();
  display.setTextColor(SSD1306_WHITE);
  display.setTextSize(1);

  // Header
  display.setCursor(0, 0);
  display.print("2048");
```

```
  display.setCursor(40, 0);
  display.print("Score:");
  display.print(score);

  // Grid (de y=12 a y=56 -> 44px total)
  const int gx = 0;
  const int gy = 12;
  const int cellW = 32;   // 128/4
  const int cellH = 11;   // 44/4

  // líneas
  for (int i=0; i<=4; i++) {
    display.drawLine(gx, gy + i*cellH, gx + 4*cellW, gy + i*cellH, SSD1306_WHITE);
    display.drawLine(gx + i*cellW, gy, gx + i*cellW, gy + 4*cellH, SSD1306_WHITE);
  }

  // números (simple, cabe siempre)
  for (int r=0; r<4; r++) {
    for (int c=0; c<4; c++) {
      uint16_t e = board[r][c];
      if (e == 0) continue;

      uint16_t v = expToValue(e);
      int x = gx + c*cellW + 2;
      int y = gy + r*cellH + 2;

      display.setCursor(x, y);
      display.print(v);
    }
  }

  // barra inferior
  display.fillRect(0, 56, 128, 8, SSD1306_BLACK);
  display.setCursor(0, 56);
  display.print("2^ 8v 4< 6>  9=R");

  display.display();
}

void showNoMove() {
  display.fillRect(70, 0, 58, 10, SSD1306_BLACK);
  display.setCursor(70, 0);
  display.print("NO MOVE");
  display.display();
  delay(120);
  drawBoard();
}

void showGameOver() {
  display.clearDisplay();
  display.setTextColor(SSD1306_WHITE);
```

```
  display.setTextSize(2);
  display.setCursor(16, 16);
  display.print("GAME");
  display.setCursor(16, 36);
  display.print("OVER");

  display.setTextSize(1);
  display.setCursor(0, 0);
  display.print("Press 9 to restart");
  display.display();
}

// ================= Lógica 2048 =================
bool slideAndMerge(uint16_t line[4]) {
  bool moved = false;

  // compactar
  uint16_t tmp[4] = {0,0,0,0};
  int idx = 0;
  for (int i=0; i<4; i++) if (line[i] != 0) tmp[idx++] = line[i];
  for (int i=0; i<4; i++) { if (line[i] != tmp[i]) moved = true; line[i] = tmp[i]; }

  // fusionar
  for (int i=0; i<3; i++) {
    if (line[i] != 0 && line[i] == line[i+1]) {
      line[i]++;
      line[i+1] = 0;
      score += expToValue(line[i]);
      moved = true;
      i++;
    }
  }

  // compactar otra vez
  uint16_t tmp2[4] = {0,0,0,0};
  idx = 0;
  for (int i=0; i<4; i++) if (line[i] != 0) tmp2[idx++] = line[i];
  for (int i=0; i<4; i++) { if (line[i] != tmp2[i]) moved = true; line[i] = tmp2[i]; }

  return moved;
}

bool moveLeft() {
  bool moved = false;
  for (int r=0; r<4; r++) {
    uint16_t line[4] = {board[r][0], board[r][1], board[r][2], board[r][3]};
    if (slideAndMerge(line)) moved = true;
    for (int c=0; c<4; c++) board[r][c] = line[c];
  }
  return moved;
}
```

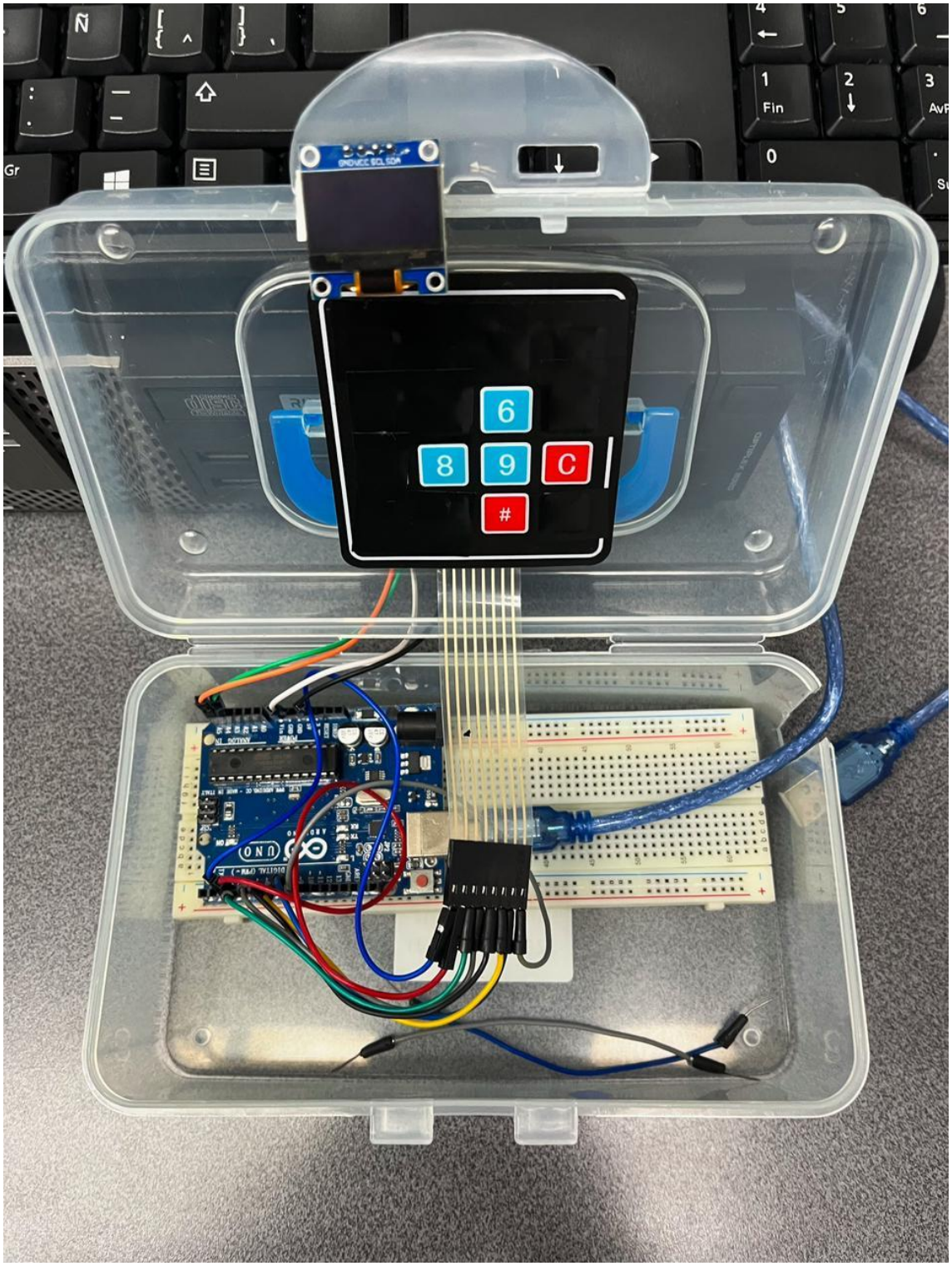
```
bool moveRight() {
  bool moved = false;
  for (int r=0; r<4; r++) {
    uint16_t line[4] = {board[r][3], board[r][2], board[r][1], board[r][0]};
    if (slideAndMerge(line)) moved = true;
    board[r][3]=line[0]; board[r][2]=line[1]; board[r][1]=line[2]; board[r][0]=line[3];
  }
  return moved;
}

bool moveUp() {
  bool moved = false;
  for (int c=0; c<4; c++) {
    uint16_t line[4] = {board[0][c], board[1][c], board[2][c], board[3][c]};
    if (slideAndMerge(line)) moved = true;
    for (int r=0; r<4; r++) board[r][c] = line[r];
  }
  return moved;
}

bool moveDown() {
  bool moved = false;
  for (int c=0; c<4; c++) {
    uint16_t line[4] = {board[3][c], board[2][c], board[1][c], board[0][c]};
    if (slideAndMerge(line)) moved = true;
    board[3][c]=line[0]; board[2][c]=line[1]; board[1][c]=line[2]; board[0][c]=line[3];
  }
  return moved;
}

void newGame() {
  for (int r=0; r<4; r++)
    for (int c=0; c<4; c++)
      board[r][c] = 0;

  score = 0;
  addRandomTile();
  addRandomTile();
  drawBoard();
}

void setup() {
  Wire.begin();

  if (!display.begin(SSD1306_SWITCHCAPVCC, OLED_ADDR)) {
    while (true) {}
  }
  display.clearDisplay();
  display.display();

  // semilla random (si A0 está libre, mejor)
  rngState ^= ((uint32_t)analogRead(A0) << 16);
```

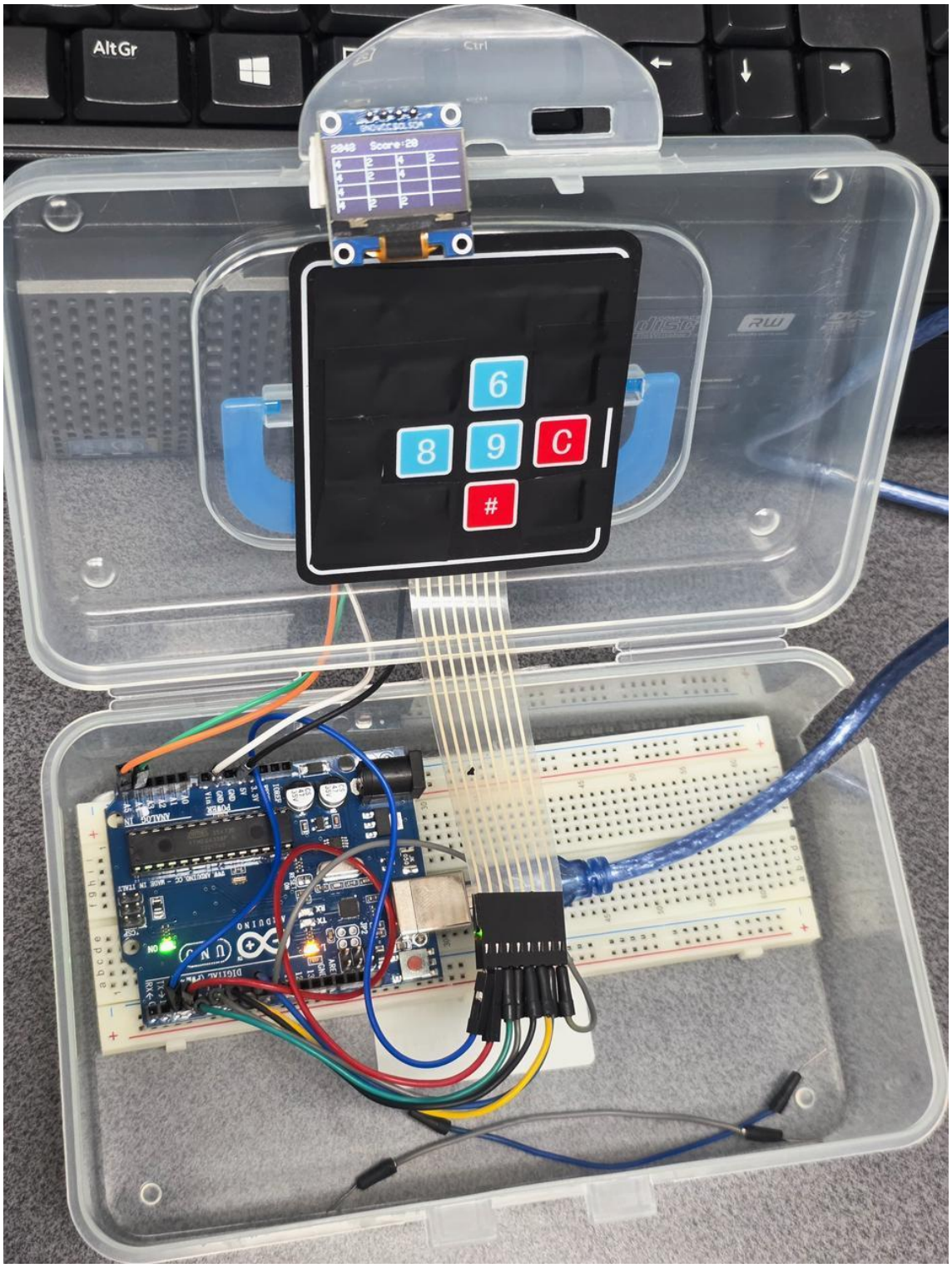
```
  rngState ^= (uint32_t)micros();

  newGame();
}

void loop() {
  char k = keypad.getKey();
  if (!k) return;

  // reset con 9
  if (k == '9') {
    newGame();
    return;
  }

  // solo 4 teclas
  bool moved = false;
  if      (k == '2') moved = moveUp();
  else if (k == '8') moved = moveDown();
  else if (k == '4') moved = moveLeft();
  else if (k == '6') moved = moveRight();
  else return;

  if (moved) {
    addRandomTile();
    drawBoard();
    if (!canMove()) showGameOver();
  } else {
    showNoMove();
  }
}
```

El código implementado en el juego “2048” en un Arduino Uno utilizando una pantalla OLED y un teclado matricial 4x4. Primero inicializa la comunicación I2C con la pantalla y configura el teclado como dispositivo de entrada. Luego crea una matriz 4x4 que representa el tablero del juego, donde cada posición almacena el valor de las fichas. En el ciclo principal, el programa detecta la tecla presionada (arriba, abajo, izquierda o derecha), ejecuta el movimiento correspondiente compactando y fusionando números iguales, actualiza el puntaje y genera un nuevo número en una posición vacía de forma pseudoaleatoria. Además, dibuja constantemente el tablero en la pantalla OLED, muestra el puntaje en tiempo real y detecta cuándo ya no hay movimientos posibles, mostrando el mensaje de “GAME OVER” o permitiendo reiniciar el juego con una tecla específica.

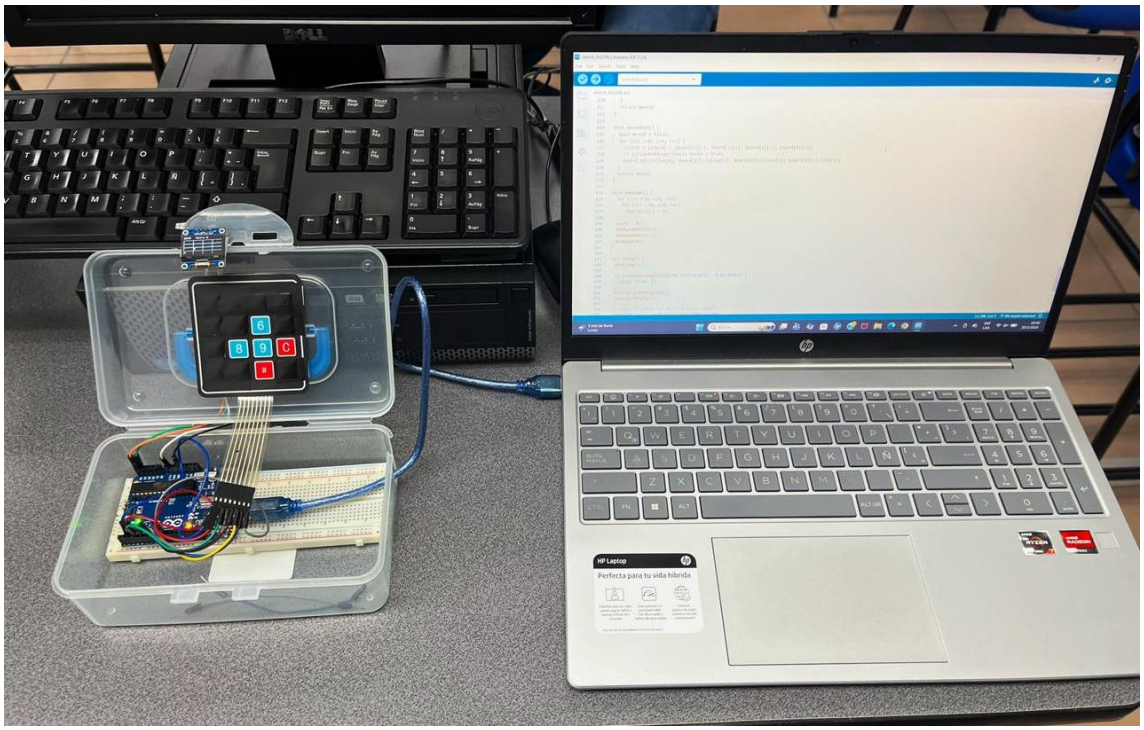
Anexos:

Montaje & simulación del juego “2048”:




AltGr
Ctrl
2048 Score:28
6
8
9
C
#

AltGr
2048
Score:16
6
8
9
C



Grupo de trabajo #1:

Evidencia experimental (Video):

Enlace:

https://youtu.be/VtdngTWqn6c

Conclusiones:

- El proyecto permitió integrar conceptos fundamentales de Arquitectura de Computadora con programación y electrónica aplicada.

- Se logró implementar correctamente un sistema interactivo completo basado en microcontrolador.

- La gestión de memoria mediante una matriz 4x4 demostró cómo estructuras simples pueden representar sistemas complejos.

- La comunicación I2C simplificó significativamente la conexión de la pantalla OLED.

- Los problemas enfrentados durante el desarrollo fortalecieron la capacidad de análisis y solución técnica del equipo.

- Se comprobó que el Arduino Uno puede manejar aplicaciones lógicas relativamente complejas sin necesidad de hardware adicional.

- El proyecto no solo cumplió con los objetivos planteados, sino que permitió experimentar de forma directa cómo la arquitectura computacional cobra vida cuando el hardware y el software trabajan en